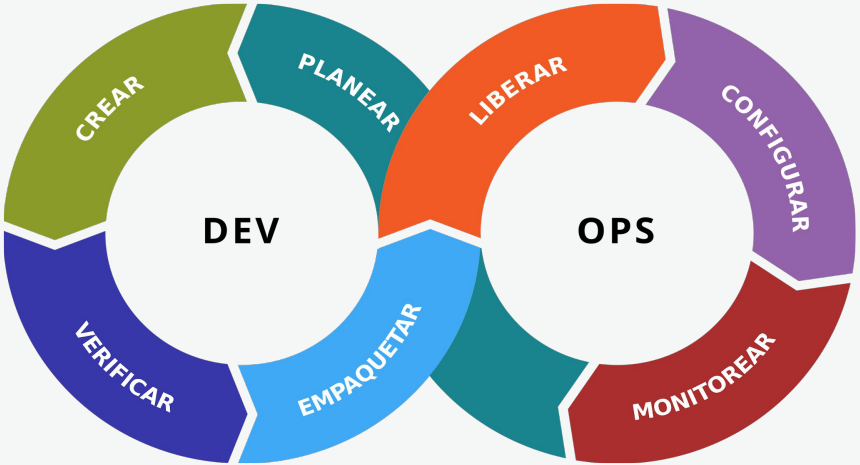
CI/CD Contd.

Topics Covered

- CI/CD
- Deployment Patterns
  - Blue/green
  - Canary
  - Ramped
- Github Action

What is CI/CD?



CI/CD stands for Continuous Integration and Continuous Delivery.

- **integration: build, package, and test applications automatically**
- **delivery: automated deployment of software with little to no downtime for users**

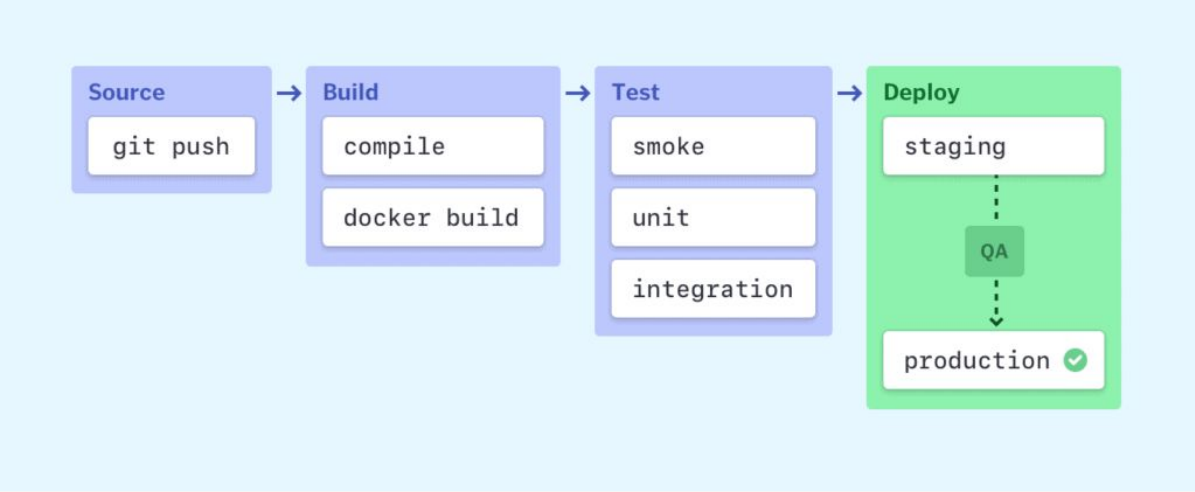
"The continuous integration/continuous delivery (CI/CD) pipeline is an agile DevOps workflow focused on a frequent and reliable software delivery process. The methodology is iterative, rather than linear, which allows DevOps teams to write code, integrate it, run tests, deliver releases and deploy changes to the software collaboratively and in real-time."

– IBM

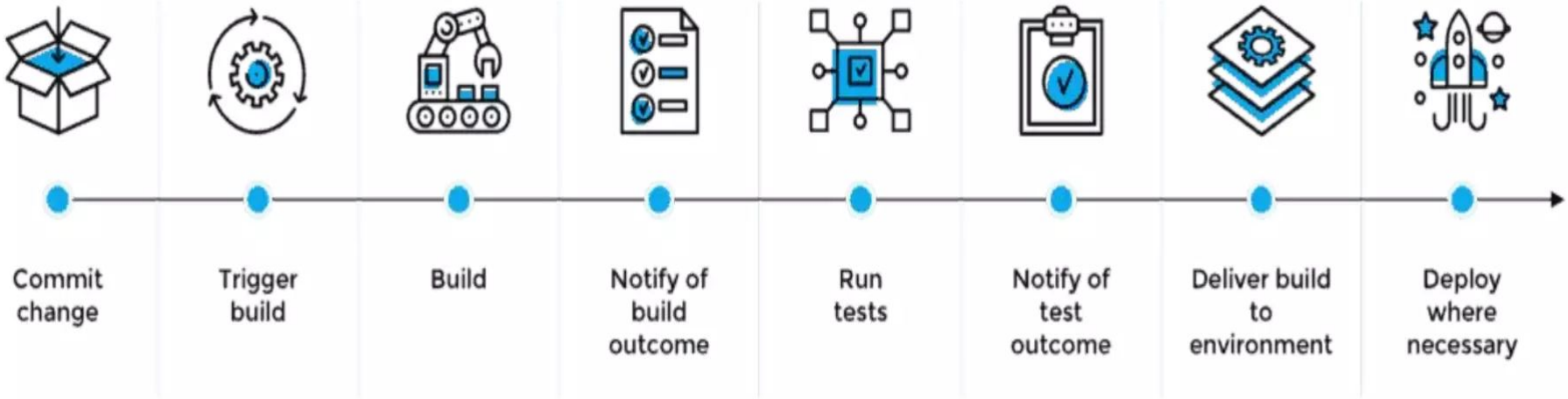
Pipelines

Pipelines describe the automated processes involved in building and deploying code.

By monitoring the status of a pipeline, we can identify issues with the build.



CI/CD pipeline




Top 20 CI/CD Tools you should consider
Buddy
GitHub Actions
Jenkins
Harness
GitLab CI
Azure DevOps
JetBrains TeamCity
Circle CI
Travis CI
JetBrains Space
BitBucket Pipelines
AWS CodeStar
Google Cloud Build
Bamboo Data Center
Drone
AppVeyor
GoCD
CodeShip
Perforce Helix
Phabricator

CI example

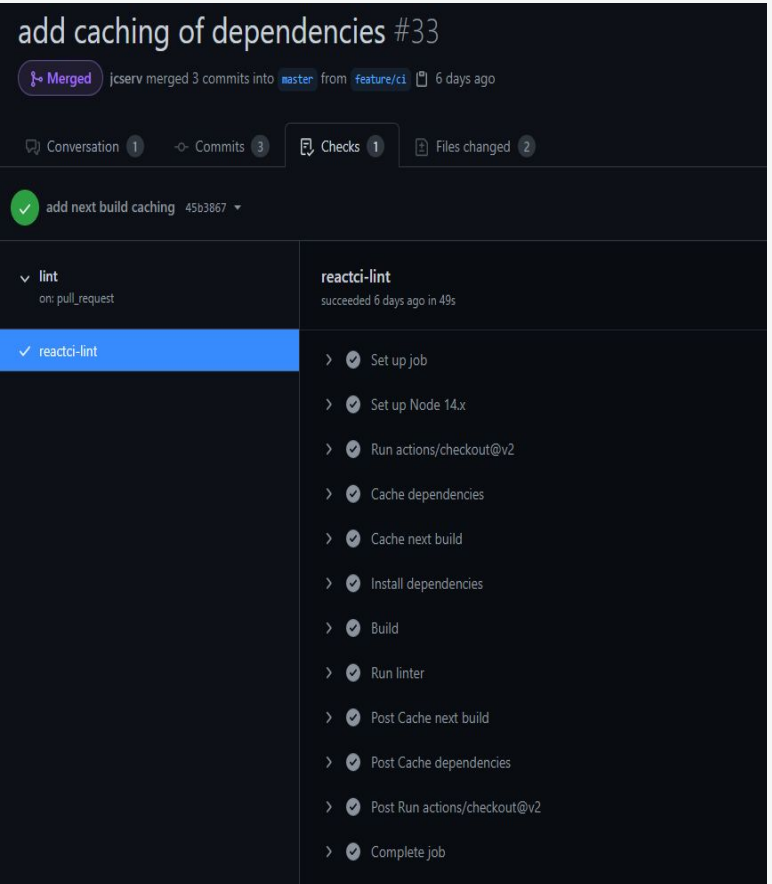
Most CI/CD work is using yaml files (e.g. jenkinsfiles, workflows, docker-compose).

This workflow:

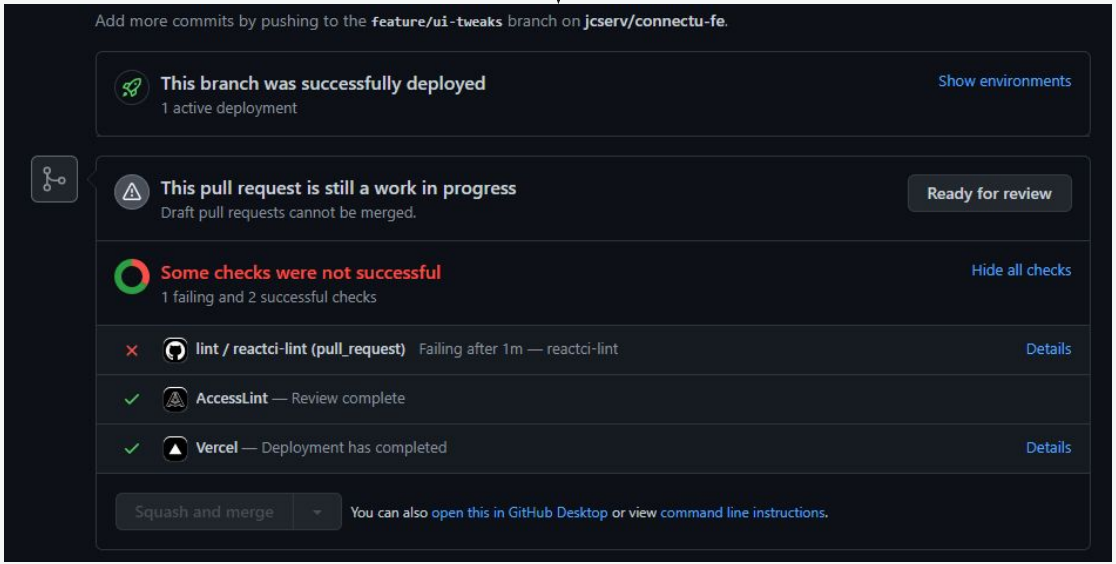
- is named “lint”
- is run on pushes to the main and develop branch and when pull requests are created
- steps:
  - sets up node environment
  - checks out the github repo
  - uses cached dependencies (if applicable)
  - uses cached build (if applicable)
  - installs dependencies if need be
  - builds the frontend application into static files
  - runs the linting command

```yaml
name: lint

on:
  push:
    branches:
      - develop
      - main
  pull_request:

jobs:
  reactci:
    name: reactci-lint
    runs-on: ubuntu-latest
    steps:
      - name: Set up Node 14.x
        uses: actions/setup-node@v1
        with:
          node-version: 14.x

      - uses: actions/checkout@v2

      - name: Cache dependencies
        uses: actions/cache@v2
        with:
          path: '**/node_modules'
          key: ${{ runner.os }}-modules-${{ hashFiles('**/yarn.lock') }}

      - name: Cache next build
        uses: actions/cache@v2
        with:
          path: ${{ github.workspace }}/.next/cache
          key: ${{ runner.os }}-nextjs-${{ hashFiles('**/yarn.lock') }}

      - name: Install dependencies
        run: yarn --prefer-offline

      - name: Build
        run: yarn run build

      - name: Run linter
        run: yarn run lint
```

CI example



if any jobs fail (ex. the linting step), then that will be visible on the pull request and merging will be prevented.



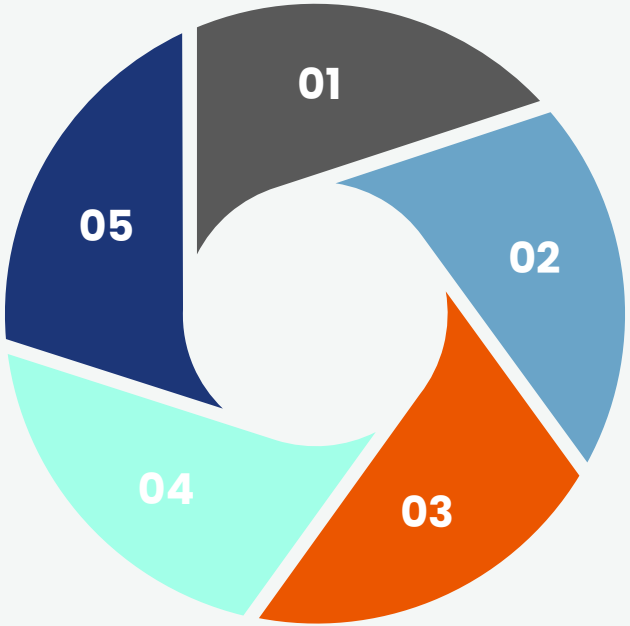
Deployment patterns

To successfully launch a new version of the software solution they provide, DevOps teams use deployment patterns. Using these techniques, network traffic in a production environment is transitioned from the old version to the new version based on the firm's specialty. Deployment patterns can influence downtime and operational costs based on the company's specialty.

Deployment patterns

Blue/Green deployment

Shadow deployment



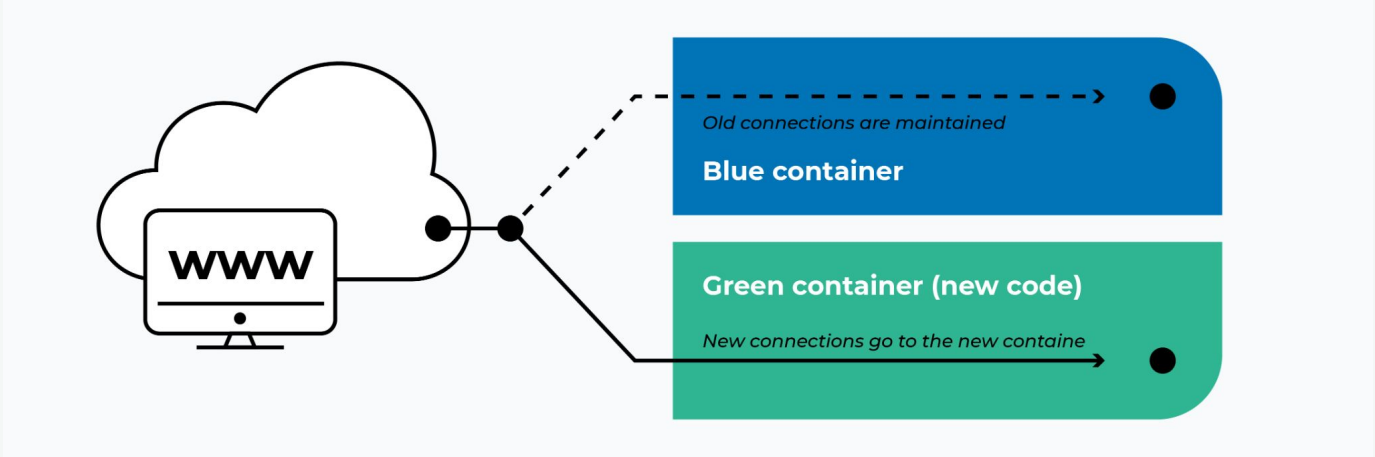
Canary deployment

Ramped deployment

A/B test deployment

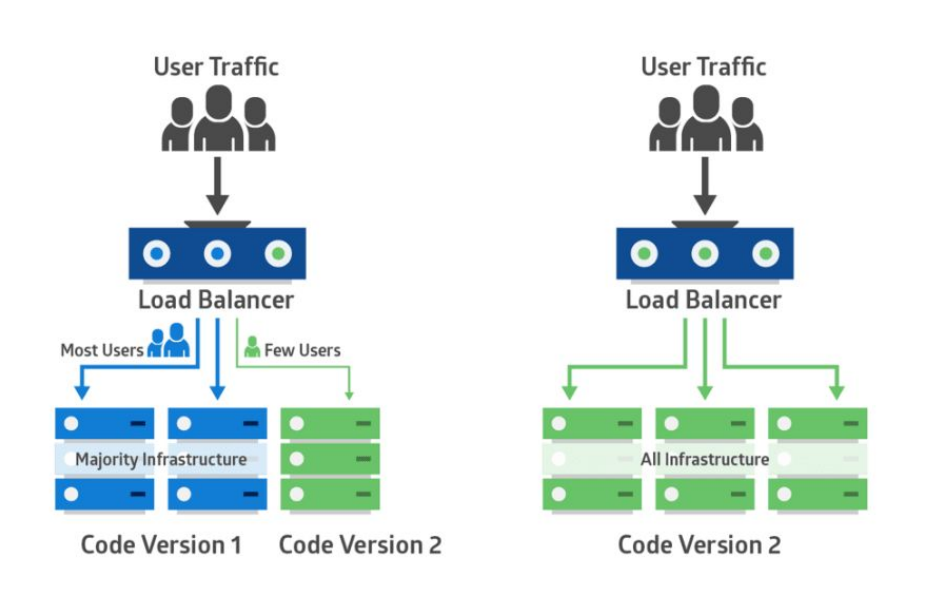
Blue/green deployment

Blue/green deployment is a deployment scheme enabling changes to be pushed while having no downtime for users. Traffic is routed to a container with the old code, while a new container is spun up with the new code. After building, the traffic is rerouted and the old container can be torn down.



Canary deployment

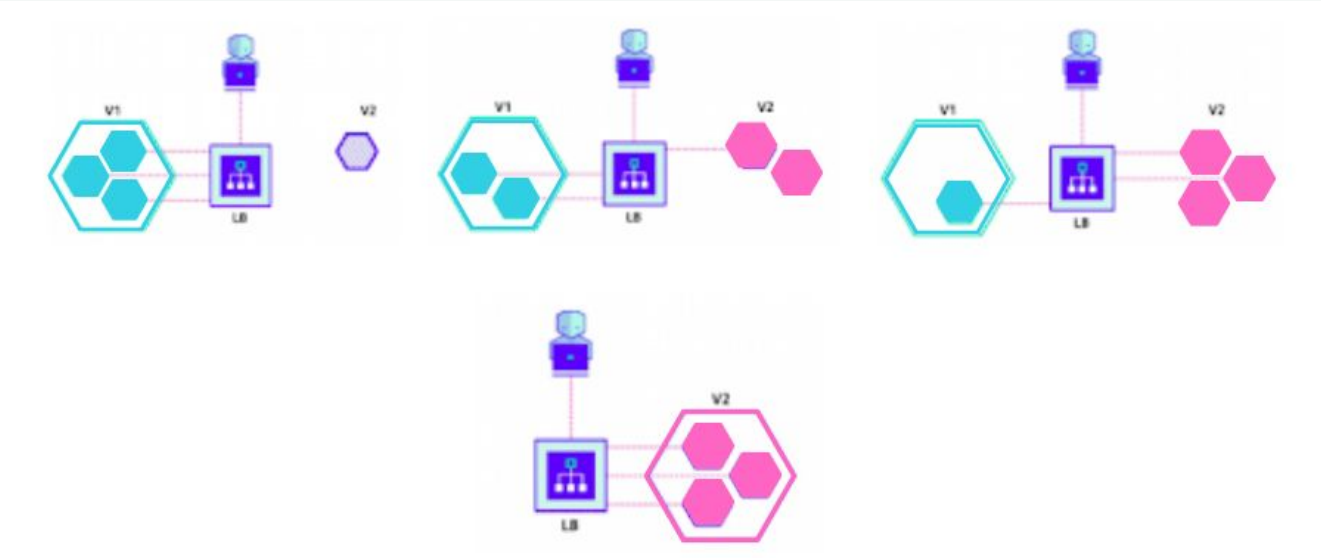
canary deployment is the practice of staging releases to release changes to a small batch of users, receive & address feedback, and then roll out changes to the entire user base gradually.



In coal mine, during the 1900s, miners would take canaries down into the tunnels with them. If dangerous gases such as carbon monoxide collected in the mine, the gases would kill the canary before killing the miners, thus providing a warning to exit the tunnels immediately.

Ramped deployment

Ramped deployment gradually changes the older version to the new version. Unlike canary deployment, the ramped deployment strategy replaces instances of the old application version with instances of the new application version one instance at a time



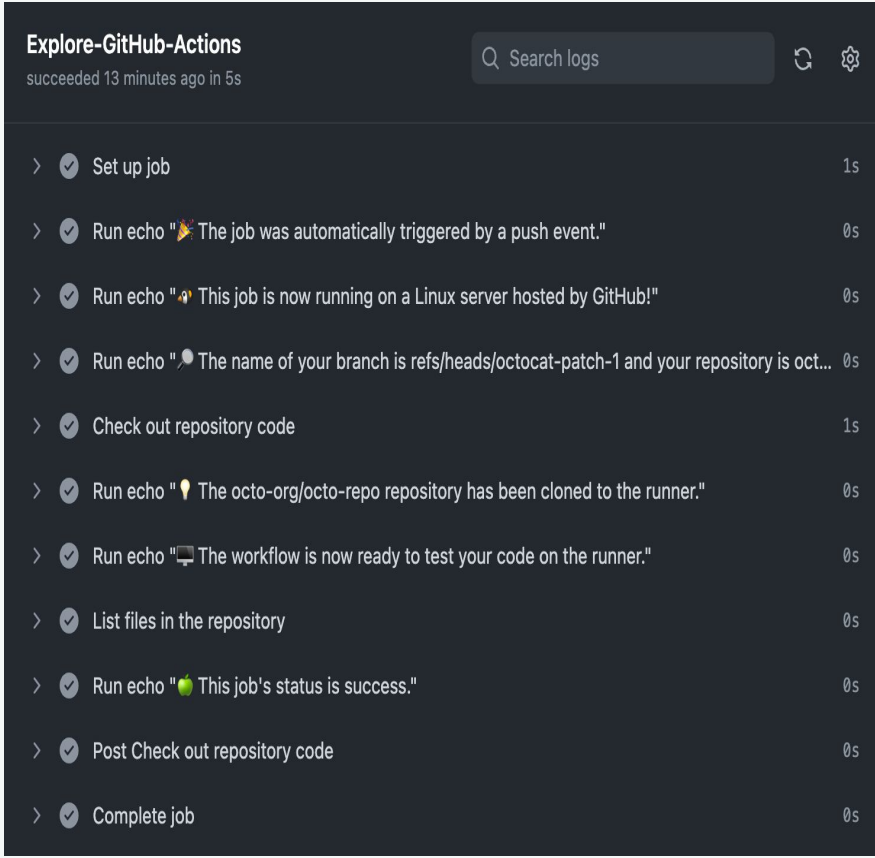
Why CI/CD?

- **Automation of the test, build & deployment process:** making the development process more efficient, reducing the length of the software delivery process, allowing a developer's changes to a cloud application to go live within minutes of writing them.
- **Continuous feedback:** Every time code is tested, developers can quickly take action on the feedback and improve the code, promoting agile & test driven development.
- **Early fault detection**: In continuous integration, testing is automated for each version of code built to look for issues integration. These issues are easier to fix the earlier in the pipeline that they occur.
- **Improving team collaboration and system integration.** Everyone on the team can change code, respond to feedback and quickly respond to any issues that occur.

GitHub Action

GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline.

You can create workflows that build and test every pull request to your repository, or deploy merged pull requests to production.



Demo Time!

1. Clone/fork the demo repo from github classroom

2. Complete the workflow

3. Go to your repo on GitHub website and play with the github actions

CI demo

For every push or pr to the main branch, it will run the workflow:

1. Sets up node environment
2. Checks out the github repo
3. Installs dependencies and test for frontend
4. Installs dependencies and test for frontend
5. Sends test coverage to some code analysis platform (optional)

ub > workflows > github-actions-workflow.yml

```yaml
name: CI/CD
run-name: ${{ github.actor }} is running ${{ github.workflow }} on ${{ github.r
on:
    push:
        branches:
            - main
    pull_request:
        branches:
            - main

jobs:
    test:
        runs-on: ubuntu-latest
        timeout-minutes: 10
        strategy:
            matrix:
                node-version: [12.x]
        steps:
            - name: Clone repository
              uses: actions/checkout@v2

            - name: Use Node.js 12.x
              uses: actions/setup-node@v1
              with:
                  node-version: 12.x

            - name: Testing frontend
              run: |
                  ls
                  cd frontend
                  npm install
                  npm test

            - name: Testing backend
              run: |
                  cd backend
                  npm install
                  npm test

            # - name: Publish code coverage
```

CD demo

1. Checks out the github repo
2. Builds the images with docker-compose
3. Testing (if applicable)
4. Login to Docker Hub with secretes
5. Publish the Docker images to your repositories on Docker Hub

You can then pull and run the published images on any deploy machines!

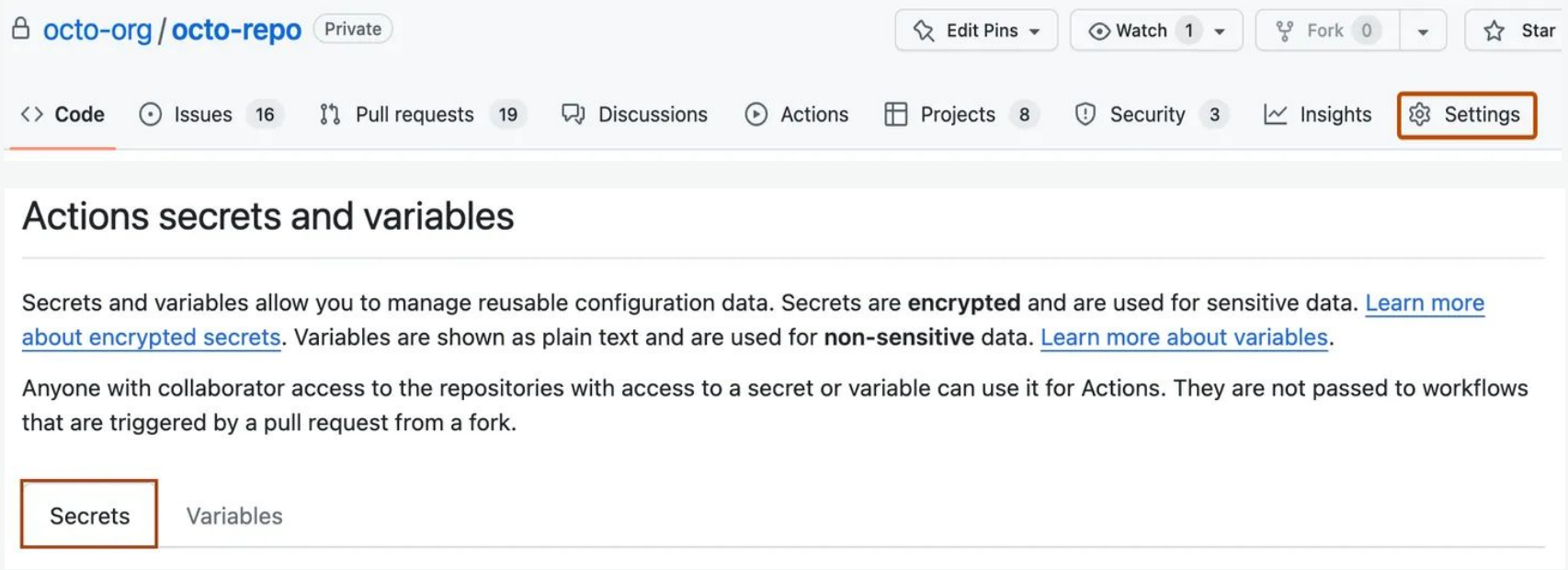
ode Blame 26 lines (23 loc) · 919 Bytes

```yaml
name: Build and deploy to Docker Hub
on:
  workflow_dispatch:
jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4

      - name: Docker compose build
        run: docker-compose build

      - name: Log in to Docker Hub
        uses: docker/login-action@f4ef78c080cd8ba55a85445d5b36e214a81df20a
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}

      - name: Push to Docker Hub
        run: |
          docker tag ci-cd-example_frontend:latest ${{ secrets.DOCKER_USERNAME }}/cicd-demo-frontend:late
          docker tag ci-cd-example_backend:latest ${{ secrets.DOCKER_USERNAME }}/cicd-demo-backend:latest
          docker push ${{ secrets.DOCKER_USERNAME }}/cicd-demo-frontend:latest
          docker push ${{ secrets.DOCKER_USERNAME }}/cicd-demo-backend:latest
          docker images
```

GitHub Secret

Development environment secrets allow you to store sensitive information in your organization or repository for use with GitHub Codespaces.

You should not reveal any password or API keys in a public (or even private) repo!!!



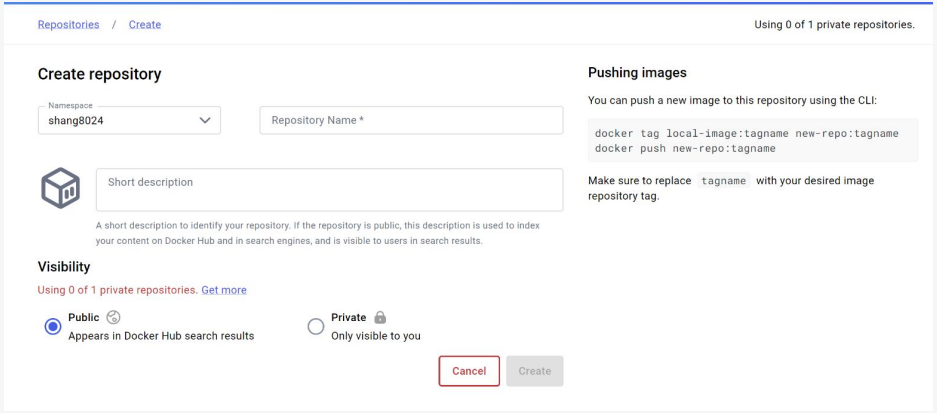
Using Secrets in Github Actions

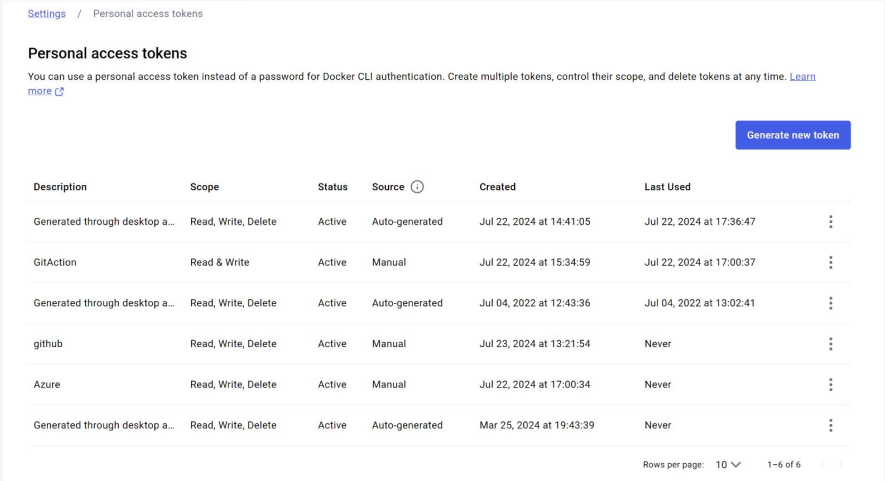
To provide an action with a secret as an input or environment variable, you can use the secrets context to access secrets you've created in your repository.

```yaml
steps:
  - name: Hello world action
    with: # Set the secret as an input
      super_secret: ${{ secrets.SuperSecret }}
    env: # Or as an environment variable
      super_secret: ${{ secrets.SuperSecret }}
```

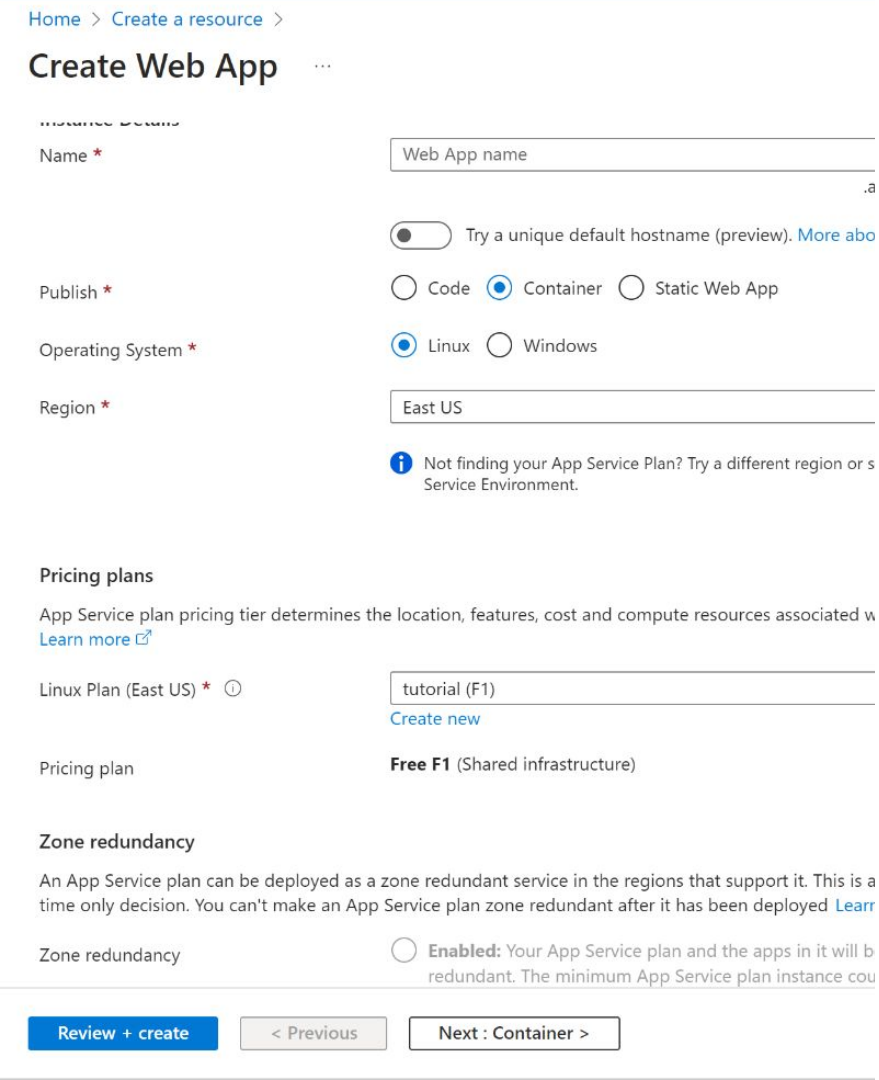
DockerHub

Docker Hub is a container registry built for developers and open source contributors to find, use, and share their container images.



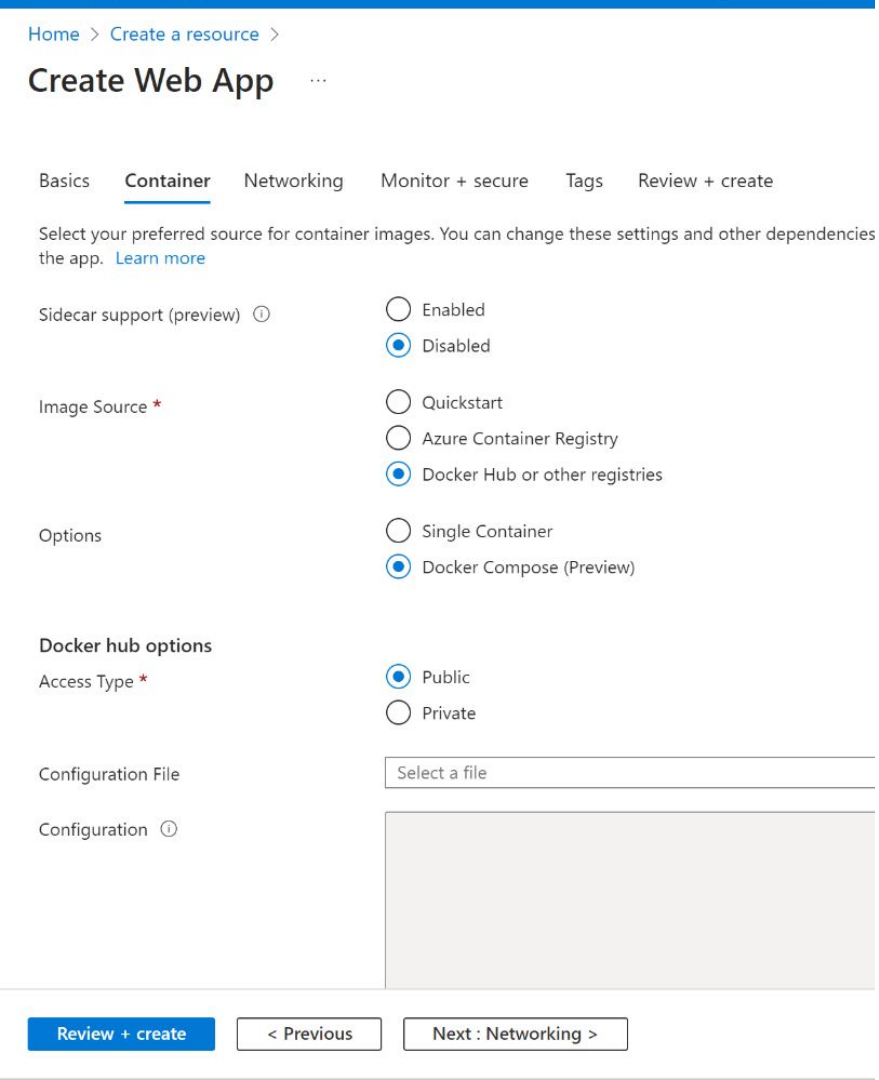


You will need to login to the dockerhub to publish the image on your login machine, or use a personal access token for CLI authentication.



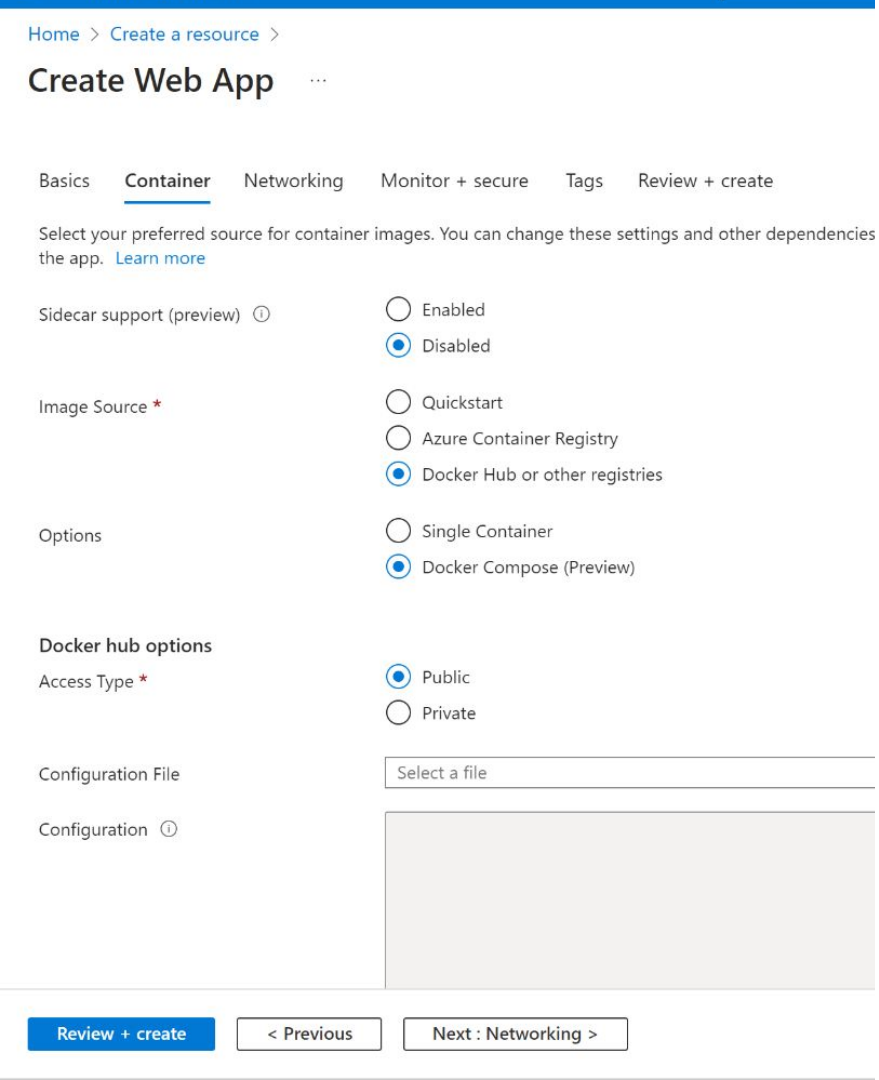
We will be using Azure for deployment this time!

1. Go to Azure web servie website, signup for a free account
2. Click on “Create a Resource” and create a “web app”
3. For instance Details, choose “Container” as Publish
4. For pricing plan, create a new one and choose free plan
5. Next+Container, choose “Docker hub” →“docker compose”→”Public”
6. Fill out the remaining required field whatever you like, and create your web app!



We will be using Azure for deployment this time!

1. Go to Azure web servie website, signup for a free account
2. Click on “Create Resource” and create a “web app”
3. For Instance Details, choose “Container” as Publish
4. For Pricing plan, create a new one and choose free plan
5. Next+Container, choose “Docker hub” →“docker compose”→”Public”
6. Fill out the remaining required field whatever you like, and create your web app!
7. After creation completed, click on “go to resource” and then go to deployment center



We will be using Azure for deployment this time!

1. Go to Azure web servie website, signup for a free account
2. Click on “Create Resource” and create a “web app”
3. For Instance Details, choose “Container” as Publish
4. For Pricing plan, create a new one and choose free plan
5. Next+Container, choose “Docker hub” →“docker compose”→”Public”
6. Fill out the remaining required field whatever you like, and create your web app!
7. After creation completed, click on “go to resource” and then go to deployment center